

Table of Contents

Internship Assignment: Build a Modern Chat Application.....	2
Duration.....	2
Objective.....	2
Technology Stack.....	3
Backend.....	3
Frontend.....	3
AI.....	4
Project.....	4
Database Requirements.....	4
Phase 1 — REST API Chat.....	5
Branch.....	5
Goal.....	5
Example APIs.....	5
Authentication (Custom + Google Auth).....	5
Groups.....	5
Messages.....	5
Concepts To Learn.....	6
Phase 2 — Polling.....	6
Branch.....	6
Goal.....	6
Expected Result.....	7
Concepts To Learn.....	7
Phase 3 — Real-Time Chat using WebSockets.....	7
Branch.....	7
Goal.....	7
Expected Result.....	8
Concepts To Learn.....	8
Phase 4 — Daily AI Chat Summary.....	8
Branch.....	8
Goal.....	8
Requirements.....	9
Concepts To Learn.....	9
Phase 5 — Distributed Workers and BullMQ Flows.....	10
Branch.....	10
Goal.....	10
Step 1 — Scheduler Queue.....	10
Step 2 — Parent Flow Jobs.....	10

Step 3 — Child Jobs.....	11
Queues.....	11
Workers.....	11
Environment Configuration.....	12
Concepts To Learn.....	12
BullMQ.....	12
Advanced Concepts.....	13
Infrastructure Concepts.....	13
Bonus Features (Optional).....	13
Submission Requirements.....	13
Final Presentation.....	14
Database.....	14
REST APIs.....	14
Polling.....	14
WebSockets.....	14
BullMQ.....	14
BullMQ Flows.....	15
AI Integration.....	15
Scaling.....	15
Final Goal.....	15

Internship Assignment: Build a Modern Chat Application

Duration

2 Weeks

Objective

The goal of this assignment is not just to build a working application.

The goal is to understand how modern applications evolve over time and how different technologies solve different problems.

You are allowed and encouraged to use AI tools, vibe coding, ChatGPT, Claude, Gemini, Cursor, Copilot, or any other tools to help you move faster.

However, if you use AI-generated code, you are expected to fully understand:

- What the code does.
- Why was it implemented that way?
- What problem it solves.
- How you would modify it.
- How you would debug it if it fails.

During the final review, you may be asked to explain any part of your implementation.

Technology Stack

Backend

- Node.js
- PostgreSQL
- Prisma ORM
- BullMQ
- Redis
- Socket.IO

Frontend

- Next.js

AI

- Google Gemini API
 - Vercel AI SDK
-

Project

Build a **Group Chat Application**.

The application should allow users to:

- Register and login
- Create chat groups
- Join groups
- Send messages
- View chat history
- Receive system-generated messages

Simple authentication is sufficient.

Database Requirements

At minimum, your application should contain:

- Users
- Groups
- Group Members
- Messages

You are free to add additional tables if needed.

You should properly design:

- Relationships
 - Foreign keys
 - Indexes
 - Constraints
-

Phase 1 — REST API Chat

Branch

`feature/rest-chat`

Goal

Build the complete chat application using only REST APIs.

When a user opens a chat room:

- Frontend requests messages from the API.
- Sending a message uses an API request.
- Users only see new messages after refreshing or reopening the chat.

Example APIs

Authentication (Custom + Google Auth)

- POST `/auth/register`
- POST `/auth/login`

Groups

- `POST /groups`
- `GET /groups`
- `GET /groups/:id`

Messages

- `POST /groups/:id/messages`
 - `GET /groups/:id/messages`
-

Concepts To Learn

- REST APIs
 - Prisma relationships
 - Database migrations
 - Authentication
 - Authorization
 - Pagination
 - Validation
 - Database indexing
-

Phase 2 — Polling

Branch

`feature/chat-polling`

Create this branch from:

`feature/rest-chat`

Goal

Improve the user experience without changing the backend architecture. The frontend should automatically fetch new messages every 10 seconds.

Possible implementations:

- `setInterval`
- React Query polling
- SWR refresh interval

Expected Result

- User opens a chat room.
 - Messages refresh every 10 seconds.
 - Users receive new messages without manually refreshing.
-

Concepts To Learn

- Polling
 - API optimization
 - Server load considerations
 - Caching strategies
 - Tradeoffs of polling
-

Phase 3 — Real-Time Chat using WebSockets

Branch

`feature/chat-websocket`

Create this branch from:

`feature/chat-polling`

Goal

Replace polling with real-time communication.

When one user sends a message, all connected users should receive it immediately.

Use: Socket.IO

Polling should no longer be used for new messages.

Expected Result

- User A sends a message.
- User B receives it instantly.
- Multiple users can chat in real time.

Concepts To Learn

- WebSocket lifecycle
 - Connection management
 - Rooms
 - Events
 - Event-driven architecture
 - Scaling challenges of WebSockets
-

Phase 4 — Daily AI Chat Summary

Branch

`feature/chat-summary`

Create this branch from:

`feature/chat-websocket`

Goal

Introduce background jobs and AI integration.

Every day, the system should generate a summary of the previous 24 hours of conversation and post it back into the chat group.

Example:

Daily Summary

- Authentication module completed.
 - Prisma migration issue resolved.
 - Frontend chat UI improved.
 - Next focus: WebSocket integration.
-

Requirements

Use:

- BullMQ
- Redis
- Gemini API with Vercel AI SDK

Create a repeatable BullMQ job:

chat-summary-scheduler

This job should run once every 24 hours.

The scheduler should:

1. Find all groups that had activity during the last 24 hours.
2. Ignore inactive groups.
3. Create a summary job for every active group.

Example query:

```
SELECT DISTINCT group_id
FROM messages
WHERE created_at >= NOW() - INTERVAL '24 HOURS';
```

Concepts To Learn

- Background jobs
 - Redis
 - Repeatable jobs
 - Retry strategies
 - AI integrations
 - Rate limiting
 - Failure handling
-

Phase 5 — Distributed Workers and BullMQ Flows

Branch

[feature/chat-summary-distributed](#) Create this branch from: [feature/chat-summary](#)

Goal

Refactor the summary system into multiple queues, workers, and parent-child jobs.

This phase introduces concepts used in real production systems.

Step 1 — Scheduler Queue

The repeatable scheduler job runs every 24 hours.

Queue: scheduler-queue

Worker:scheduler-worker

Responsibility:

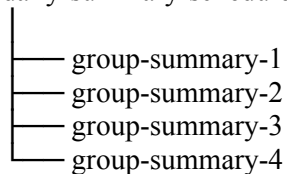
- Find active groups.
 - Create workflows for each group.
-

Step 2 — Parent Flow Jobs

For each active group create a parent job.

Example:

daily-summary-scheduler



Step 3 — Child Jobs

Each group summary should create child jobs.

Example:

group-summary-123



The parent job should only complete after all child jobs finish successfully.

Queues

Create separate queues for different work types.

Example:

```
scheduler-queue  
summary-queue  
ai-queue  
notification-queue
```

Workers

Create separate workers for each queue.

Example:

```
scheduler-worker  
summary-worker  
ai-worker  
notification-worker
```

Each worker should run independently.

Environment Configuration

Worker concurrency should be configurable using environment variables.

Example:

```
SCHEDULER_WORKER_CONCURRENCY=1  
SUMMARY_WORKER_CONCURRENCY=5  
AI_WORKER_CONCURRENCY=10  
NOTIFICATION_WORKER_CONCURRENCY=3
```

The application should read these values during startup.

This allows scaling without code changes.

Example:

Development:

AI_WORKER_CONCURRENCY=1

Production:

AI_WORKER_CONCURRENCY=20

Concepts To Learn

BullMQ

- Queue
- Worker
- Queue Events
- Delayed Jobs
- Repeatable Jobs
- Retry Strategies

Advanced Concepts

- FlowProducer
- Parent Jobs
- Child Jobs
- Dependency Trees
- Workflow Orchestration
- Distributed Processing

Infrastructure Concepts

- Horizontal scaling
- Worker isolation
- Failure isolation
- Resource allocation
- Worker concurrency

Bonus Features (Optional)

- Typing indicators
 - Read receipts
 - Message reactions
 - Online users
 - File uploads
 - Edit messages
 - Delete messages
 - Message search
-

Submission Requirements

You must maintain separate branches for every phase.

1. `feature/rest-chat`
2. `feature/chat-polling`
3. `feature/chat-websocket`
4. `feature/chat-summary`
5. `feature/chat-summary-distributed`

Each branch should build on the previous branch.

Final Presentation

Each intern should prepare a short demonstration explaining:

Database

- Table design
- Relationships
- Indexes

REST APIs

- API design decisions
- Validation
- Authentication

Polling

- How it works
- Advantages and disadvantages

WebSockets

- Event flow
- Room management
- Connection lifecycle

BullMQ

- Queues
- Workers
- Retry strategies
- Failure handling

BullMQ Flows

- Parent jobs
- Child jobs
- FlowProducer
- Workflow orchestration

AI Integration

- Prompt design
- Summary generation
- Rate limiting considerations

Scaling

- Which workers would you scale first?
 - How would you support 1 million users?
 - What bottlenecks would appear?
-

Final Goal

By the end of this assignment, you should understand how modern applications evolve from:

REST APIs → Polling → WebSockets → Background Jobs → AI Integration → Distributed Systems

If you complete all phases successfully and understand the concepts behind them, you will have experience with many of the building blocks used in production backend systems today.